

Bayero University Kano
Faculty of Computing and Information Technology
CSC 4311: Artificial Intelligence
Module 3: Problem Solving and Search

Tutors: Dr Muhammad Yusuf Muhammad, Najib Abdulrazak, Bilal Lawal Alhassan, Lawal Yusuf Abdullahi

Contents

1	Introduction to Problem Solving in AI	4
2	Problem Formulation	6
3	Search Strategies	6
4	Breadth-First Search (BFS)	7
4.1	Learning Objectives	7
4.2	Definition	7
4.3	Basic Idea	7
4.4	Illustration of BFS Search	8
4.5	Queue Operation	8
4.6	BFS Algorithm	8
4.7	Worked Example	9
4.8	Properties of BFS	9
4.9	Complexity Analysis	10
4.10	Advantages	10
4.11	Disadvantages	10
4.12	Applications	10
4.13	Real-World Example	11
5	Depth-First Search (DFS)	11
5.1	Learning Objectives	11
5.2	Definition	11

5.3	Basic Idea	12
5.4	How DFS Works	12
5.5	Illustration of DFS	12
5.6	Stack Operation	13
5.7	Backtracking	13
5.8	DFS Algorithm	13
5.9	Worked Example	14
5.10	Properties of DFS	14
5.11	Complexity Analysis	14
5.12	Advantages	15
5.13	Disadvantages	15
5.14	Applications	15
5.15	Real-World Example	15
5.16	Comparison Between BFS and DFS	16
6	Depth-Limited Search (DLS)	17
7	Iterative Deepening Search (IDS)	18
8	Greedy Best-First Search (GBFS)	19
8.1	Learning Objectives	19
8.2	Definition	20
8.3	Intuition Behind GBFS	20
8.4	Heuristic Function	20
8.5	Evaluation Function	21
8.6	Illustrative Search Tree	21
8.7	Algorithm	22
8.8	Worked Example	22
8.9	Complexity Analysis	23
8.10	Advantages	23
8.11	Disadvantages	23
8.12	Applications	24
8.13	Comparison with UCS and A*	24
8.14	Real-World Example	24
9	A* Search	24
9.1	Learning Objectives	25
9.2	Motivation	25
9.3	Why A* Search?	26
9.4	Definition	26
9.5	Understanding the Evaluation Function	26
9.6	Illustration of the Evaluation Function	28
9.7	Numerical Example	28
9.8	Intuition Behind A* Search	29
9.9	Comparison of Search Strategies	29
9.10	Key Points	29
9.11	How A* Search Works	30
9.12	Algorithm	30
9.13	Worked Example	31

9.14 Step 1	31
9.15 Step 2	31
9.16 Step 3	32
9.17 Step 4	33
9.18 Step 5	33
9.19 Summary of Node Values	33
9.20 Search Progress	33
9.21 Observations	33
10 Search Performance Evaluation	34
11 Applications of Search in AI	34

1 Introduction to Problem Solving in AI

Problem solving is a cornerstone of Artificial Intelligence (AI). Many intelligent behaviors—such as playing games, navigating maps, diagnosing medical issues, or planning a sequence of tasks—can be modeled as **search problems**, where the goal is to determine a sequence of actions that leads to a desired state.

In AI, a **problem** refers to a computational task that can be solved using search algorithms. It is defined by a set of formal components that specify the starting point, the goal, and the means to reach that goal.

Definition of a Problem in AI

A **problem** in AI is formally defined as a tuple that includes:

- **Initial state** — the situation or configuration from which the agent starts.
- **Actions** — the set of all valid operations the agent can perform.
- **Transition model** — a description of how actions change the environment.
- **Goal test** — a condition that checks whether the goal has been achieved.
- **Path cost** — the cost associated with reaching a state through a series of actions.

Together, these components form a **state space**, which represents all possible configurations that can be reached from the initial state.

Components of a Problem

- **Initial State:** The beginning state of the problem.
- **Actions (Operators):** The set of legal moves available to the agent.
- **Transition Model:** Describes what each action does. It defines the outcome of applying an action to a state.
- **Goal Test:** A function that returns true if the current state matches the goal state.
- **Path Cost:** The numerical cost of a path from the initial state to a goal. This helps in optimizing the solution.

Example: 8-Puzzle

The 8-puzzle consists of a 3x3 grid with tiles numbered 1 through 8 and one empty space. The goal is to arrange the tiles in a specific target configuration using sliding moves.

- **Initial state:** Random arrangement of tiles.
- **Actions:** Slide a tile up, down, left, or right into the blank space.
- **Goal test:** Tiles match the desired end state.
- **Path cost:** Number of moves or time taken.

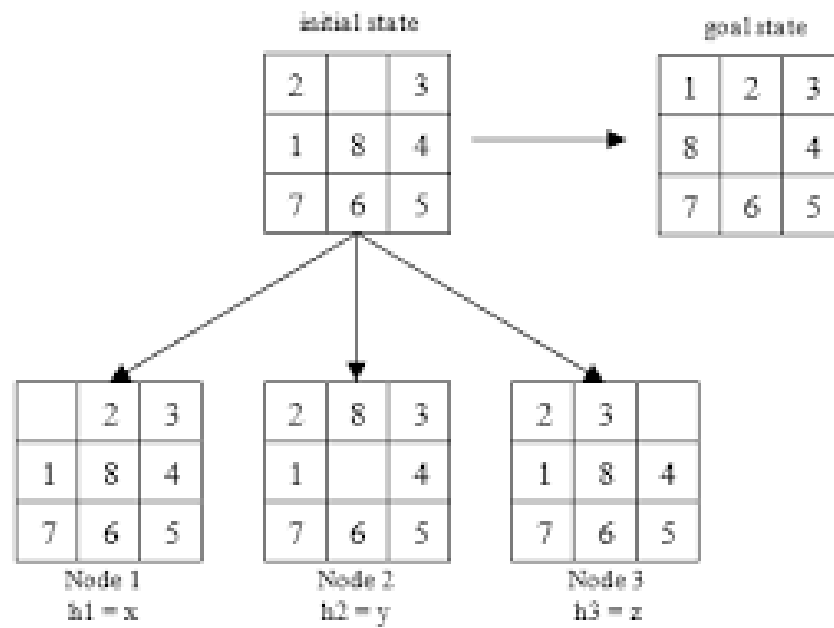


Figure 1: 8-Puzzle Problem

Example: Route-Finding Problem

In this problem, the agent must find the optimal route between two cities.

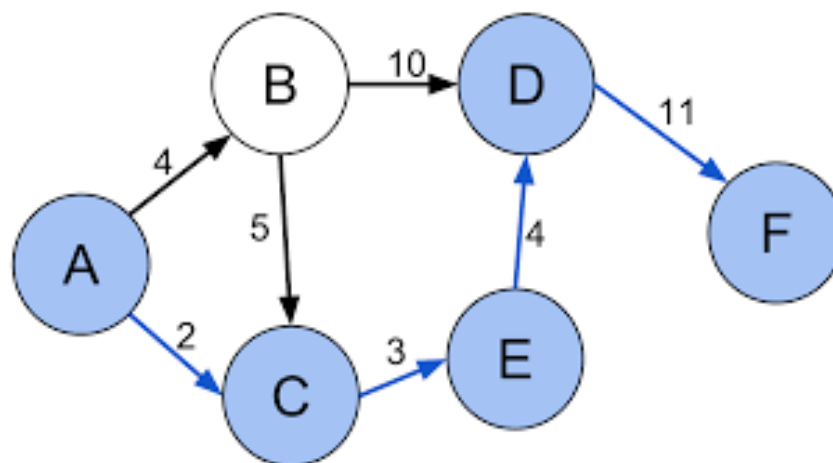


Figure 2: 8-P

- **Initial state:** Start at city A.
- **Actions:** Move to adjacent cities via roads.
- **Transition model:** Taking road from one city to another changes location.
- **Goal test:** Arrived at city F.
- **Path cost:** Total distance or travel time.

2 Problem Formulation

Well-Defined vs Ill-Defined Problems

- **Well-Defined Problem:** Has a clear initial state, goal state, and set of operations. Example: Chess.
- **Ill-Defined Problem:** Lacks clarity in one or more components. Example: vague goals or undefined transitions

Example:

- **Well-Defined:** Solving a maze where the start and end are known.
- **Ill-Defined:** Writing an essay on "What is a good life?"

Example: Vacuum Cleaner World

An environment where a vacuum agent must clean a 2-square room. It must decide whether to move left, right, or suck dirt based on its location and sensors. The initial state could be: "Agent at left square, dirt at both squares." The goal is to clean both squares.

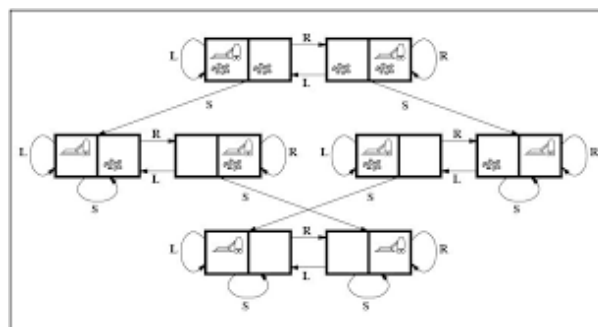


Figure 3: Vacuum Cleaner World Environment

3 Search Strategies

Search strategies are methods used by agents to explore the search space and find solutions to problems. They are generally categorized into uninformed (blind) and informed (heuristic) search strategies.

Uninformed Search (Blind Search)

These strategies do not have additional information about states beyond what is provided in the problem definition.

4 Breadth-First Search (BFS)

Breadth-First Search (BFS) is one of the most fundamental uninformed search algorithms in Artificial Intelligence. It systematically explores the search space level by level, beginning from the initial state. Instead of following one path deeply, BFS first visits every node at the current depth before proceeding to the next level.

Because BFS explores nodes in increasing order of depth, it always finds the shortest path (measured by the number of edges) when every action has the same cost.

4.1 Learning Objectives

After studying this section, students should be able to:

- Explain the concept of Breadth-First Search.
- Describe how BFS explores a search tree.
- Apply BFS to solve simple search problems.
- Implement BFS using a queue.
- Analyze the time and space complexity of BFS.
- Compare BFS with other uninformed search algorithms.

4.2 Definition

Breadth-First Search (BFS) is an uninformed search strategy that expands the shallowest unexplored node first.

It searches every node at one depth before moving to nodes at the next depth.

BFS uses a **First-In-First-Out (FIFO)** queue to store nodes waiting to be explored.

4.3 Basic Idea

Suppose an agent is searching for a goal.

Instead of moving deep into one branch,
BFS searches

Level 0

then

Level 1

then

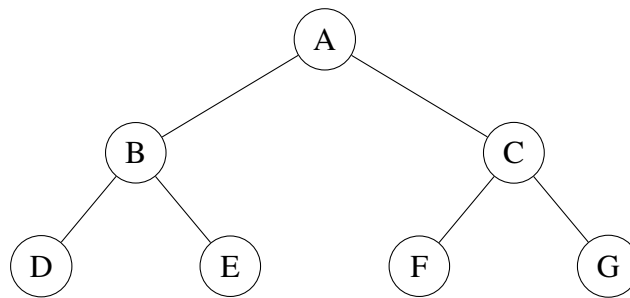
Level 2

then

Level 3

until the goal is found.

4.4 Illustration of BFS Search



BFS visits the nodes in the following order

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$$

Notice that all nodes on one level are visited before moving to the next level.

4.5 Queue Operation

The frontier in BFS is maintained using a queue.

First In $\longrightarrow$ First Out

Example

Queue
A

Expand A

Queue becomes

B
C

Expand B

Queue becomes

C
D
E

The oldest node is always removed first.

4.6 BFS Algorithm

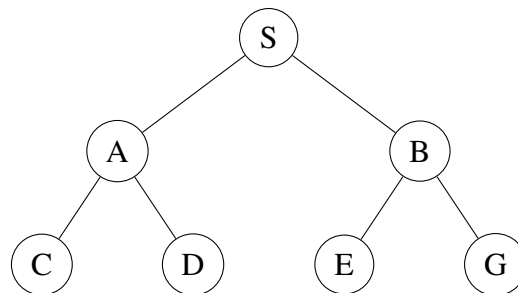
The algorithm proceeds as follows.

1. Create an empty queue.
2. Insert the initial node into the queue.
3. Repeat until the queue becomes empty.

- (a) Remove the front node.
- (b) Check whether it is the goal.
- (c) If yes, terminate.
- (d) Otherwise expand the node.
- (e) Insert all unexplored children into the rear of the queue.

4.7 Worked Example

Consider the following search tree.



Suppose the goal node is

G

The search proceeds as follows.

Step	Queue
1	S
2	A,B
3	B,C,D
4	C,D,E,G
5	D,E,G
6	E,G
7	G
8	Goal Found

4.8 Properties of BFS

Property	Description
Strategy	Level-order exploration
Complete	Yes
Optimal	Yes (equal step cost)
Data Structure	Queue (FIFO)
Search Type	Uninformed

Table 1: Properties of Breadth-First Search

4.9 Complexity Analysis

Suppose

b = branching factor

d = depth of the shallowest goal

Then

$$\text{Time Complexity} = O(b^d)$$

because every node up to depth d may be generated.

Similarly,

$$\text{Space Complexity} = O(b^d)$$

since BFS stores all frontier nodes in memory.

This makes BFS memory intensive for large search spaces.

4.10 Advantages

- Complete.
- Guaranteed to find a solution if one exists.
- Finds the shortest path when edge costs are equal.
- Easy to understand and implement.
- Suitable for shallow search problems.

4.11 Disadvantages

- Requires large memory.
- Becomes slow for large branching factors.
- Inefficient when the goal is very deep.
- Explores many unnecessary nodes.

4.12 Applications

Breadth-First Search has many practical applications.

- Route finding.
- Social network analysis.
- Web crawling.
- Broadcasting in computer networks.

- Robot navigation.
- Solving puzzles such as the 8-puzzle.
- Shortest path in unweighted graphs.

4.13 Real-World Example

Suppose a robot is searching for the nearest charging station inside a warehouse.

The robot starts from its current location and searches every nearby location first before moving farther away.

This guarantees that the robot reaches the nearest charging station using the minimum number of movements.

5 Depth-First Search (DFS)

Depth-First Search (DFS) is one of the most widely used uninformed search algorithms in Artificial Intelligence. Unlike Breadth-First Search (BFS), which explores nodes level by level, DFS explores one branch of the search tree as deeply as possible before backtracking to explore alternative branches.

DFS is particularly useful when the solution is expected to be located deep within the search tree or when memory resources are limited. Because it stores only the nodes along the current search path, DFS requires significantly less memory than BFS.

5.1 Learning Objectives

After studying this section, students should be able to:

- Define Depth-First Search.
- Explain the principle behind DFS.
- Describe how a stack is used in DFS.
- Trace the execution of DFS on a search tree.
- Analyze the advantages and disadvantages of DFS.
- Compare DFS with BFS.
- Implement DFS using recursion or a stack.

5.2 Definition

Depth-First Search (DFS) is an uninformed search strategy that always expands the deepest unexplored node first.

Instead of exploring all neighbouring nodes, DFS follows one path until

- the goal is found,
- a dead end is reached, or

- the maximum search depth is reached.

When a dead end is encountered, the algorithm returns to the nearest unexplored ancestor and continues searching. This process is known as **backtracking**.

5.3 Basic Idea

Consider a student searching for a book in a library.

Instead of checking one shelf from every row (as BFS would do), the student walks to the end of one row, checking every shelf before returning to another row.

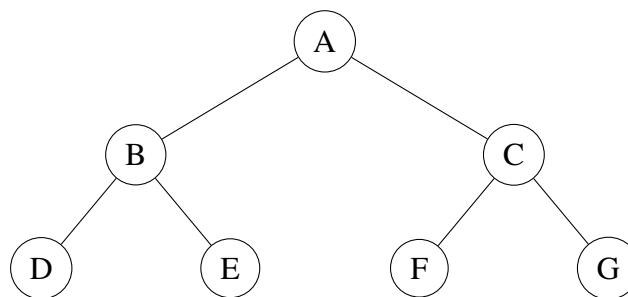
This is exactly how DFS explores a search tree.

5.4 How DFS Works

The algorithm follows these steps:

1. Start at the initial node.
2. Push the node onto a stack.
3. Pop the top node.
4. Check whether it is the goal.
5. If not, push its unexplored children onto the stack.
6. Repeat until the goal is found or the stack becomes empty.

5.5 Illustration of DFS



If the left child is expanded first, DFS visits

$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$$

Observe that DFS explores one complete branch before moving to another.

5.6 Stack Operation

DFS uses a Last-In-First-Out (LIFO) stack.

The most recently generated node is expanded first.

Last In $\longrightarrow$ First Out

Example:

Initial stack

[A]

Expand A

[C, B]

Expand B

[C, E, D]

Expand D

[C, E]

Expand E

[C]

Expand C

[G, F]

5.7 Backtracking

One of the distinguishing characteristics of DFS is backtracking. Suppose the algorithm reaches node D and discovers that D has no children. Instead of terminating, the algorithm returns to node B and explores the remaining child E. Backtracking allows DFS to systematically explore every possible path.

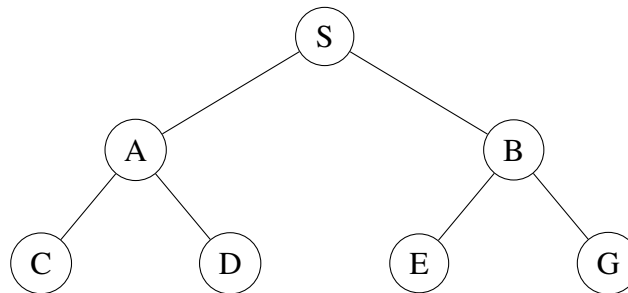
5.8 DFS Algorithm

The algorithm is summarized below.

1. Create an empty stack.
2. Push the initial state.
3. While the stack is not empty
 - (a) Pop the top node.
 - (b) If it is the goal, stop.
 - (c) Otherwise push all unexplored children.
4. If the stack becomes empty, return failure.

5.9 Worked Example

Consider the following search tree.



Suppose G is the goal.

The search proceeds as follows.

$$S \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow E \rightarrow G$$

Notice that DFS explores an entire branch before considering other branches.

5.10 Properties of DFS

Property	Description
Search Strategy	Deepest Node First
Data Structure	Stack
Complete	No (unless finite tree)
Optimal	No
Memory Usage	Low
Search Type	Uninformed

Table 2: Properties of Depth-First Search

5.11 Complexity Analysis

Let

b

be the branching factor
and

m

be the maximum depth.
Then

Time Complexity = $O(b^m)$

because DFS may explore every node.
The memory requirement is

$$\text{Space Complexity} = O(bm)$$

This is much smaller than BFS.

5.12 Advantages

- Requires less memory than BFS.
- Easy to implement.
- Suitable when solutions are deep.
- Can be implemented recursively.
- Useful for backtracking problems.

5.13 Disadvantages

- Does not guarantee the shortest path.
- May become trapped in infinite paths.
- Can miss shallow solutions.
- Sensitive to node ordering.

5.14 Applications

Depth-First Search is widely used in many AI and Computer Science applications.

- Maze solving.
- Puzzle solving.
- Game tree exploration.
- Topological sorting.
- Cycle detection.
- Robot navigation.
- Web crawling.
- Constraint Satisfaction Problems (CSPs).

5.15 Real-World Example

Suppose a rescue robot is searching for survivors inside a collapsed building.

Instead of searching every room on the first floor before moving to the second floor, the robot follows one corridor until it reaches the end. If no survivor is found, it returns to the previous junction and explores another corridor.

This behaviour closely resembles Depth-First Search.

5.16 Comparison Between BFS and DFS

Feature	BFS	DFS
Data Structure	Queue	Stack
Search Order	Level-by-Level	Deepest First
Optimal	Yes	No
Complete	Yes	No
Memory Usage	High	Low
Shortest Path	Yes	No
Implementation	Easy	Easy

Table 3: Comparison of BFS and DFS

Uniform Cost Search: Expands the node with the lowest path cost. Useful when costs vary.

Uniform Cost Search (UCS)

Uniform Cost Search (UCS) expands the node with the lowest **cumulative path cost** from the root. It is ideal when the cost of actions varies.

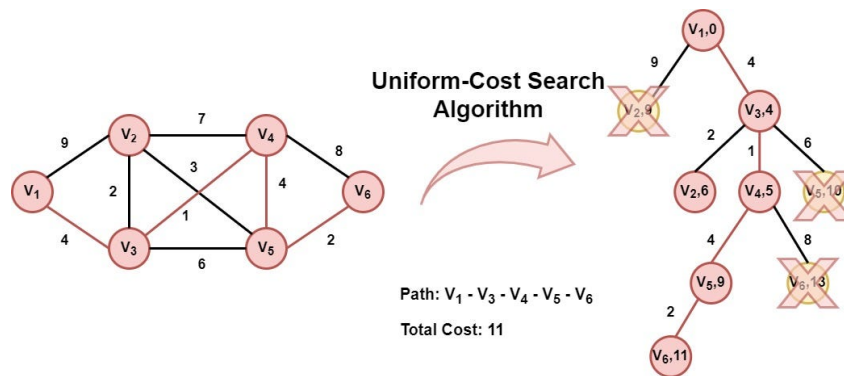


Figure 4: Uniform cost search Problem

Key Characteristics: Uses a **priority queue**, ordered by total path cost $g(n)$. **Completeness:** Yes – will always find a solution if one exists. **Optimality:** Yes – finds the lowest-cost solution. **Time Complexity:** $O(b^{1+\frac{C^*}{\epsilon}})$ where C^* is the cost of the optimal solution, and ϵ is the minimum action cost. **Space Complexity:** Can be high due to storing explored paths.

Example: Delivery Routing Problem

Imagine a delivery robot navigating a road network where some paths have tolls or different fuel costs. UCS will expand the cheapest total-cost path first, not necessarily the shortest in steps.

Steps:

1. Start from the initial node.
2. Insert it into a priority queue with cost 0.
3. Always expand the node with the lowest path cost.

4. If it is the goal, stop; otherwise, enqueue successors with updated cumulative costs.

Advantages:

- Guarantees optimal solution (lowest cost).
- Handles variable cost environments well.

Disadvantages:

- Can be slow in large or infinite spaces.
- Memory-intensive: stores many paths and their costs.

.

6 Depth-Limited Search (DLS)

Depth-Limited Search (DLS) is a variation of Depth-First Search (DFS) that introduces a predefined depth limit to avoid infinite paths in deep or unbounded trees.

Key Idea

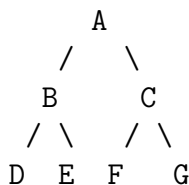
DLS explores nodes of a search tree up to a specified depth limit l . It cuts off the search at that depth, preventing infinite descent in the case of cycles or infinite branches.

Algorithm Behavior

- Starts from the root node.
- Recursively explores each branch up to the given depth limit.
- Returns ‘cutoff’ if the goal is not found within the depth.

Example

Suppose you’re navigating a tree of cities from City A (root) to reach City G:



- With depth limit $l = 1$, only A, B, and C are explored. G will not be found.
- With $l = 2$, the algorithm finds G.

Advantages

- Prevents infinite loops in DFS.
- Less memory usage compared to BFS.
- Allows partial exploration of deep trees.

Disadvantages

- Can miss the goal if depth limit is too small.
 - Requires a good estimate of goal depth.
 - May repeat work if goal is deeper than expected.
- **Iterative Deepening Search:** Repeatedly applies depth-limited search with increasing limits.

7 Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS) is a strategy that combines the space-efficiency of Depth-First Search (DFS) with the completeness of Breadth-First Search (BFS). It does this by performing a series of depth-limited searches, starting from a depth of 0 and incrementing the limit with each iteration until the goal is found.

How it Works

1. Perform a Depth-Limited Search (DLS) with a limit of 0.
2. If the goal is not found, increase the limit by 1.
3. Repeat the DLS until the goal is found or the maximum limit is reached.

Although IDS revisits nodes multiple times, this is computationally inexpensive for most practical problems because the number of nodes increases exponentially with depth, so most time is spent at the deepest level.

Advantages

- Complete: Guaranteed to find a solution if one exists.
- Optimal: Finds the shallowest (and often cheapest) solution when all actions have the same cost.
- Space-Efficient: Requires only $O(b \times d)$ memory, where b is the branching factor and d is the depth.

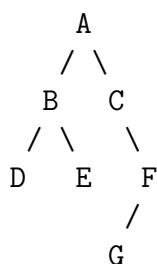
Disadvantages

- Repeatedly explores the same nodes at shallower depths, leading to redundant work.
- Less efficient for problems with large depth and high branching factors.

Time and Space Complexity

- Time Complexity: $O(b^d)$
- Space Complexity: $O(b \times d)$

Example: Binary Tree Search Suppose an agent must find the node G in the following binary tree:



- Depth 0: Visit A
- Depth 1: Visit A, B, C
- Depth 2: Visit A, B, C, D, E, F
- Depth 3: Visit A, B, C, D, E, F, G (Goal found)

Informed Search (Heuristic Search)

Informed search strategies use domain-specific knowledge (heuristics) to estimate how close a given state is to the goal. These strategies aim to reduce the number of states explored by guiding the search process using heuristics.

8 Greedy Best-First Search (GBFS)

Greedy Best-First Search (GBFS) is an informed (heuristic) search algorithm that selects the node that appears to be closest to the goal according to a heuristic function. Unlike Uniform Cost Search (UCS), which considers the actual cost of reaching a node, Greedy Best-First Search ignores the path cost and focuses only on the estimated remaining distance to the goal. The term **greedy** refers to the algorithm's strategy of making the locally optimal choice at each step in the hope of finding a globally optimal solution.

8.1 Learning Objectives

After studying this section, students should be able to: Define Greedy Best-First Search. Explain the concept of heuristic search. Understand the role of heuristic functions. Apply GBFS to solve search problems. Distinguish GBFS from UCS and A* Search. Analyze the strengths and weaknesses of GBFS.

8.2 Definition

Greedy Best-First Search is an informed search algorithm that always expands the node with the smallest heuristic value.

The evaluation function is

$$f(n) = h(n)$$

where

- $h(n)$ = estimated cost from node n to the goal.

Unlike A* Search,

GBFS completely ignores

$$g(n)$$

which is the cost already incurred.

8.3 Intuition Behind GBFS

Imagine you are driving from Kano to Lagos.

Instead of considering

- the total distance already travelled,
- fuel consumed,
- toll charges,

you simply choose the road that appears to move you closest to Lagos.

This is exactly the idea behind Greedy Best-First Search.

The algorithm always chooses the node that appears closest to the goal according to the heuristic estimate.

8.4 Heuristic Function

A heuristic function estimates how far a node is from the goal.

It is represented as

$$h(n)$$

Examples include

- Straight-line distance.
- Manhattan distance.

- Euclidean distance.
- Number of misplaced tiles (8-puzzle).
- Manhattan distance in the 8-puzzle.

A good heuristic should

- be easy to compute,
- guide the search toward the goal,
- reduce unnecessary node expansions.

8.5 Evaluation Function

Greedy Best-First Search evaluates each node using

$$f(n) = h(n)$$

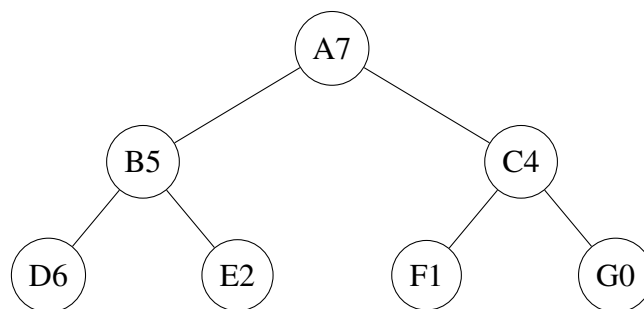
Only the heuristic estimate is considered.

The path cost

$$g(n)$$

is ignored.

8.6 Illustrative Search Tree



The second number inside each node represents

$$h(n)$$

Greedy Best-First Search always expands the node with the smallest heuristic value.

8.7 Algorithm

The algorithm proceeds as follows.

1. Insert the initial node into the priority queue.
2. Compute its heuristic value.
3. Remove the node with the smallest heuristic value.
4. If it is the goal, terminate.
5. Otherwise expand the node.
6. Compute heuristic values for all successors.
7. Insert successors into the priority queue.
8. Repeat until the goal is found.

8.8 Worked Example

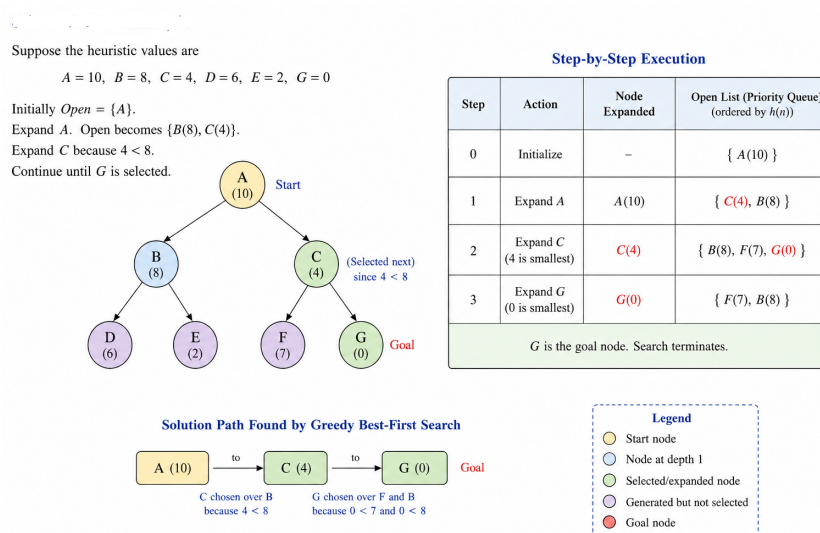


Figure 5: GBFS example

Notice that the algorithm always follows the node with the lowest heuristic estimate.

Table 4: Properties of Greedy Best-First Search

Property	Description
Search Type	Informed Search
Evaluation Function	$f(n) = h(n)$
Data Structure	Priority Queue
Complete	No (may fail in infinite spaces)
Optimal	No
Memory Usage	High
Uses Heuristic	Yes

8.9 Complexity Analysis

Let

$$b$$

be the branching factor

and

$$m$$

the maximum search depth.

Worst-case complexity is

$$Time = O(b^m)$$

Space complexity is also

$$O(b^m)$$

because all frontier nodes are stored. However, when a good heuristic is available, the actual number of explored nodes is usually much smaller.

8.10 Advantages

- Often much faster than uninformed search.
- Simple to implement.
- Uses domain knowledge.
- Can quickly reach the goal when the heuristic is accurate.
- Suitable for large search spaces.

8.11 Disadvantages

- Not guaranteed to find the shortest path.
- May become trapped in local minima.
- Strongly depends on the quality of the heuristic.
- Can revisit nodes.
- Not complete in infinite search spaces.

8.12 Applications

Greedy Best-First Search is used in

- GPS navigation.
- Robot navigation.
- Video game pathfinding.
- Network routing.
- AI planning.
- Puzzle solving.
- Medical diagnosis systems.

8.13 Comparison with UCS and A*

Table 5: Comparison of UCS, GBFS and A* Search

Feature	UCS	GBFS	A*
Uses Path Cost $g(n)$	Yes	No	Yes
Uses Heuristic $h(n)$	No	Yes	Yes
Evaluation Function	$g(n)$	$h(n)$	$g(n) + h(n)$
Optimal	Yes	No	Yes*
Complete	Yes	No	Yes*

Provided the heuristic used by A is admissible (and, for graph search, consistent).

8.14 Real-World Example

Suppose a drone must deliver medicine to a rural hospital.

Using satellite data,

the drone always flies toward the location that appears closest to the destination.

Although this strategy usually reaches the destination quickly,

it may not choose the shortest or safest route if mountains or restricted airspace exist.

9 A* Search

A* (pronounced "A-Star") Search is one of the most important informed search algorithms in Artificial Intelligence. It combines the advantages of Uniform Cost Search (UCS) and Greedy Best-First Search (GBFS) by considering both the actual cost of reaching a node and the estimated cost from that node to the goal.

Unlike Greedy Best-First Search, which only considers the estimated distance to the goal, A* also accounts for the cost already incurred. This balance enables A* to find an optimal solution while often expanding far fewer nodes than Uniform Cost Search.

Today, A* Search is widely used in robotics, autonomous vehicles, video games, GPS navigation systems, logistics, and many other intelligent systems.

9.1 Learning Objectives

After studying this section, students should be able to:

- Explain the motivation behind the development of A* Search.
- Define A* Search and describe how it works.
- Explain the meaning of the evaluation function.
- Distinguish between $g(n)$, $h(n)$ and $f(n)$.
- Apply A* Search to solve path-finding problems.
- Explain why A* is complete and optimal under suitable conditions.

9.2 Motivation

In previous sections, we studied two important search algorithms:

- Uniform Cost Search (UCS)
- Greedy Best-First Search (GBFS)

Although both algorithms are useful, each has certain limitations.

Limitations of Uniform Cost Search

Uniform Cost Search always expands the node with the smallest accumulated path cost. Its evaluation function is

$$f(n) = g(n)$$

where

$$g(n)$$

is the cost from the start node to node n .

Although UCS always finds the optimal solution, it ignores useful information about the location of the goal. As a result, it may expand many unnecessary nodes before reaching the destination.

Limitations of Greedy Best-First Search

Greedy Best-First Search uses only the heuristic estimate

$$f(n) = h(n)$$

where

$$h(n)$$

estimates the remaining cost to the goal.

Since it ignores the cost already travelled, it may choose a path that appears promising but is actually much more expensive.

Consequently, Greedy Best-First Search is generally not optimal.

9.3 Why A* Search?

A* Search was developed to overcome the limitations of both UCS and GBFS.

Instead of considering only one type of information, A* combines:

1. the actual cost from the start node, and
2. the estimated cost to the goal.

Consequently, A* can intelligently balance exploration and exploitation.

9.4 Definition

A* Search is an informed search algorithm that always expands the node with the smallest estimated total cost of reaching the goal.

The estimated total cost is computed using the evaluation function

$$f(n) = g(n) + h(n)$$

where

- $g(n)$ = actual path cost from the start node to node n ,
- $h(n)$ = estimated cost from node n to the goal,
- $f(n)$ = estimated total cost of a solution passing through node n .

9.5 Understanding the Evaluation Function

The evaluation function

$$f(n) = g(n) + h(n)$$

is the heart of the A* algorithm.

Each component has a specific meaning.

1. Path Cost $g(n)$

The function

$$g(n)$$

represents the exact cost incurred in travelling from the initial state to node n .

For example,

if travelling

$$A \rightarrow B \rightarrow C$$

costs

$$3 + 5 = 8,$$

then

$$g(C) = 8.$$

2. Heuristic Cost $h(n)$

The heuristic function

$$h(n)$$

estimates the remaining cost from node n to the goal.

Unlike $g(n)$,

this value is only an estimate.

Examples include

- Straight-line distance.
- Manhattan distance.
- Euclidean distance.
- Number of misplaced tiles.

3. Estimated Total Cost $f(n)$

The function

$$f(n)$$

is the estimated total cost of a complete solution.

It is obtained by adding

$$g(n)$$

and

$$h(n).$$

Therefore,

$$f(n) = g(n) + h(n)$$

9.6 Illustration of the Evaluation Function

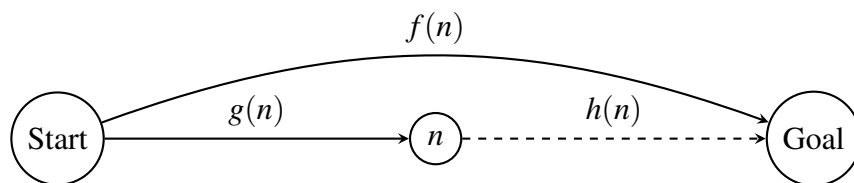


Figure above illustrates the three quantities used in A* Search.

- The solid arrow represents the actual cost already travelled ($g(n)$).
- The dashed arrow represents the estimated remaining cost ($h(n)$).
- The curved arrow represents the estimated total solution cost ($f(n)$).

9.7 Numerical Example

Suppose the search has reached node B .

The cost from the start node to B is

$$g(B) = 7.$$

The heuristic estimate from B to the goal is

$$h(B) = 5.$$

Then

$$f(B) = g(B) + h(B)$$

$$= 7 + 5$$

$$= 12.$$

Therefore,
the estimated total cost of reaching the goal through node B is

$$\boxed{12}.$$

9.8 Intuition Behind A* Search

Imagine that you are travelling from Kano to Abuja.

To choose the best route, you should consider two pieces of information:

1. How far you have already travelled.
2. Approximately how far you still have to travel.

If you only consider the remaining distance, you may choose a route with expensive tolls or heavy traffic.

If you only consider the distance already travelled, you may ignore a much shorter route ahead.

A* combines both pieces of information, enabling it to make more informed decisions than either Uniform Cost Search or Greedy Best-First Search alone.

9.9 Comparison of Search Strategies

Table 6: Comparison of Evaluation Functions

Algorithm	Evaluation Function
Uniform Cost Search	$f(n) = g(n)$
Greedy Best-First Search	$f(n) = h(n)$
A* Search	$f(n) = g(n) + h(n)$

9.10 Key Points

- A* is an informed search algorithm.
- It combines the strengths of UCS and Greedy Best-First Search.
- The evaluation function is $f(n) = g(n) + h(n)$.
- $g(n)$ is the actual path cost.
- $h(n)$ is the heuristic estimate to the goal.
- $f(n)$ is the estimated total solution cost.
- Nodes with the smallest $f(n)$ are expanded first.

9.11 How A* Search Works

A* Search maintains two important lists throughout the search process.

- **OPEN List:** contains nodes that have been generated but not yet expanded.
- **CLOSED List:** contains nodes that have already been expanded.

Initially, only the start node is placed in the OPEN list.

At every iteration, A* performs the following operations:

1. Select the node having the smallest value of

$$f(n) = g(n) + h(n)$$

2. Remove the node from the OPEN list.
3. Add the node to the CLOSED list.
4. Expand the node.
5. Compute the values of

$$g(n), h(n), f(n)$$

for every successor.

6. Insert the successors into the OPEN list.
7. Repeat until the goal node is selected.

9.12 Algorithm

The A* Search algorithm can be summarized as follows.

1. Create an empty OPEN list.
2. Create an empty CLOSED list.
3. Insert the start node into OPEN.
4. While OPEN is not empty
 - (a) Select the node having the smallest

$$f(n) = g(n) + h(n)$$

- (b) Remove it from OPEN.
- (c) If it is the goal node,
return the solution.
- (d) Otherwise,
expand the node.
- (e) Compute

$$g(n), h(n), f(n)$$

for each successor.

- (f) Insert unexplored successors into OPEN.
- (g) Move the expanded node into CLOSED.
5. If OPEN becomes empty,
return FAILURE.

9.13 Worked Example

Consider the weighted graph shown in Figure 6.

The heuristic value of each node is shown inside parentheses.

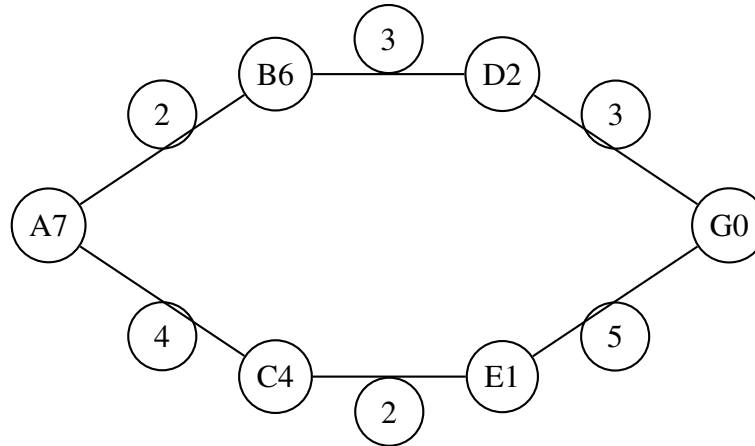


Figure 6: Weighted graph for illustrating A* Search. The number inside each node is the heuristic value $h(n)$.

9.14 Step 1

Initially,
only the start node exists.

$$OPEN = \{A\}$$

$$CLOSED = \emptyset$$

For node A

$$g(A) = 0$$

$$h(A) = 7$$

$$f(A) = 0 + 7 = 7$$

9.15 Step 2

Expand node A.
Its successors are
B and C.

For node B

$$g(B) = 2$$

$$h(B) = 6$$

$$f(B) = 2 + 6 = 8$$

For node C

$$g(C) = 4$$

$$h(C) = 4$$

$$f(C) = 4 + 4 = 8$$

The OPEN list becomes

$$\{B(8), C(8)\}$$

9.16 Step 3

Assume B is selected.

Expand node B.

Successor

D

has

$$g(D) = 5$$

$$h(D) = 2$$

$$f(D) = 7$$

Now

$$OPEN = \{D(7), C(8)\}$$

9.17 Step 4

Expand D.

Generate

Goal.

For Goal

$$g(G) = 8$$

$$h(G) = 0$$

$$f(G) = 8$$

OPEN becomes

$$\{C(8), G(8)\}$$

9.18 Step 5

Since

Goal

has been selected,

the search terminates.

The solution path is

$$A \rightarrow B \rightarrow D \rightarrow G$$

The total path cost is

$$2 + 3 + 3 = 8.$$

9.19 Summary of Node Values

9.20 Search Progress

9.21 Observations

From the worked example, the following observations can be made.

1. A* always expands the node having the smallest value of

$$f(n) = g(n) + h(n).$$

Table 7: Values of $g(n)$, $h(n)$ and $f(n)$

Node	$g(n)$	$h(n)$	$f(n)$
A	0	7	7
B	2	6	8
C	4	4	8
D	5	2	7
E	6	1	7
G	8	0	8

Table 8: Evolution of the OPEN and CLOSED Lists

Step	Expanded Node	OPEN List	CLOSED List
1	A	{A}	$\emptyset$
2	A	{B, C}	{A}
3	B	{D, C}	{A, B}
4	D	{C, G}	{A, B, D}
5	G	Goal Found	{A, B, D, G}

- Both the actual path cost and the estimated remaining cost influence the search.
- The heuristic helps A* avoid expanding many unnecessary nodes.
- Compared with Uniform Cost Search, fewer nodes are expanded.
- Compared with Greedy Best-First Search, the solution obtained is more reliable because path cost is also considered.

10 Search Performance Evaluation

To assess the effectiveness of search strategies, we consider the following criteria:

- Time Complexity:** How long the algorithm takes to find a solution.
- Space Complexity:** How much memory the algorithm uses.
- Optimality:** Whether the algorithm guarantees the best solution.
- Completeness:** Whether the algorithm always finds a solution if one exists.

11 Applications of Search in AI

Search techniques have wide applications in various AI domains:

- Route Planning:** Systems like Google Maps use A* and other algorithms to find optimal paths.
- Game Playing:** AI uses search strategies to determine optimal moves in games like Chess or Tic Tac Toe.
- Puzzle Solving:** Puzzles like Sudoku or Rubik's Cube are solved using state-space search.